

RY-SLAM: A robust YOLO-based real-time visual SLAM solution for dynamic environments

Xiaohong Chen

Academy of Military Sciences
Beijing, China
xiaohongchen04@163.com

Siwen Li

Southeast University
Nanjing, China
siwenli@seu.edu.cn

Shuangyin Ren

Academy of Military Sciences
Beijing, China
renshuangyin@126.com

Jingchao Wang*

Academy of Military Sciences
Beijing, China
wangjc.2000@tsinghua.org.cn

Abstract—Visual Simultaneous Localization and Mapping (SLAM) algorithms have been extensively utilized in applications such as intelligent navigation, autonomous driving, and mapping tasks. However, existing methods exhibit limited robustness in dynamic environments, often struggling to distinguish between static and dynamic elements effectively. To overcome this challenge, we propose Robust YOLO SLAM (RY-SLAM), a novel robust visual SLAM algorithm based on ORB-SLAM3 integrated with the Yolov8s-seg semantic segmentation network. Specifically, we develop a dedicated semantic segmentation module based on the YOLO architecture, enabling the SLAM system to accurately recognize and handle dynamic objects within the environment. Additionally, the proposed framework employs multi-threading and GPU acceleration techniques to satisfy the real-time performance requirements essential for visual SLAM applications. Consequently, the proposed approach resolves the compatibility issues traditionally encountered by visual SLAM algorithms in dynamic scenarios. Experimental evaluations performed on the TUM RGB-D dataset demonstrate that RY-SLAM significantly reduces the average absolute trajectory error (ATE) by 89.59% compared with ORB-SLAM3, highlighting its superior accuracy and robust performance in dynamic environments.

Index Terms—Visual SLAM, Dynamic Environment, YOLO

I. INTRODUCTION

Visual Simultaneous Localization and Mapping (SLAM) plays a critical role in autonomous navigation, human-computer interaction, and robotic control. Due to its scalability and lower sensor costs compared to LiDAR-based approaches, visual SLAM has attracted growing interest in recent years [1]–[4]. However, most existing visual SLAM algorithms [5], [6] have been designed and evaluated under static environmental conditions, limiting their applicability in dynamic settings such as crowded malls or busy streets, where tracking failures often occur [7], [8].

Two key criteria for evaluating visual SLAM systems are tracking robustness and real-time performance [9]. To improve robustness in dynamic environments, recent work has explored integrating neural networks—particularly those effective in image classification, detection, and segmentation—into SLAM frameworks [10]–[13]. These approaches often use semantic cues to filter dynamic keypoints during feature matching and pose estimation. While effective, such integration typically imposes high computational costs, degrading real-time performance.

This trade-off presents a fundamental challenge: accurate semantic segmentation improves robustness but slows down processing, while fast object detection models maintain real-time operation at the expense of accuracy. To address this, we propose RY-SLAM, a robust visual SLAM system that integrates the YOLOv8 object detection network with ORB-SLAM3. Through a multi-threaded tracking architecture and semantic segmentation enhancements, RY-SLAM achieves both real-time performance and improved robustness in dynamic environments. The main contributions of this paper are summarized as follows:

- 1) We introduce a parallelized semantic segmentation framework tailored for ORB-SLAM3, enabling efficient integration of the YOLOv8 network to enhance dynamic object handling without compromising system performance.
- 2) To resolve compatibility issues between ORB-SLAM3 (C++) and YOLOv8 (Python), we employ ONNXRuntime with GPU acceleration and graph optimization. A warm-up mechanism is also incorporated to reduce inference latency, significantly boosting real-time capabilities.
- 3) By extending YOLOv8 from coarse bounding boxes to fine-grained segmentation masks, our architecture enhances dynamic object detection. A neighborhood-based filtering algorithm is further introduced to refine feature point selection near mask boundaries, improving tracking stability.

II. RELATED WORK

A. Visual SLAM in Static Environments

Visual SLAM enables simultaneous localization and mapping using camera-equipped systems in unknown environments [9], [14]. Over the past decade, a variety of algorithms have been developed to tackle challenges posed by diverse environmental conditions [5], [15], [16].

ORB-SLAM2, a feature-based method proposed by Mur-Artal et al. [7], supports monocular, stereo, and RGB-D inputs. It uses ORB feature extraction [17] and inter-frame matching for pose estimation and mapping. However, its performance degrades in texture-scarce environments where feature extraction is unreliable.

In contrast, direct methods such as LSD-SLAM [15] and DTAM [18] estimate camera motion by minimizing photometric

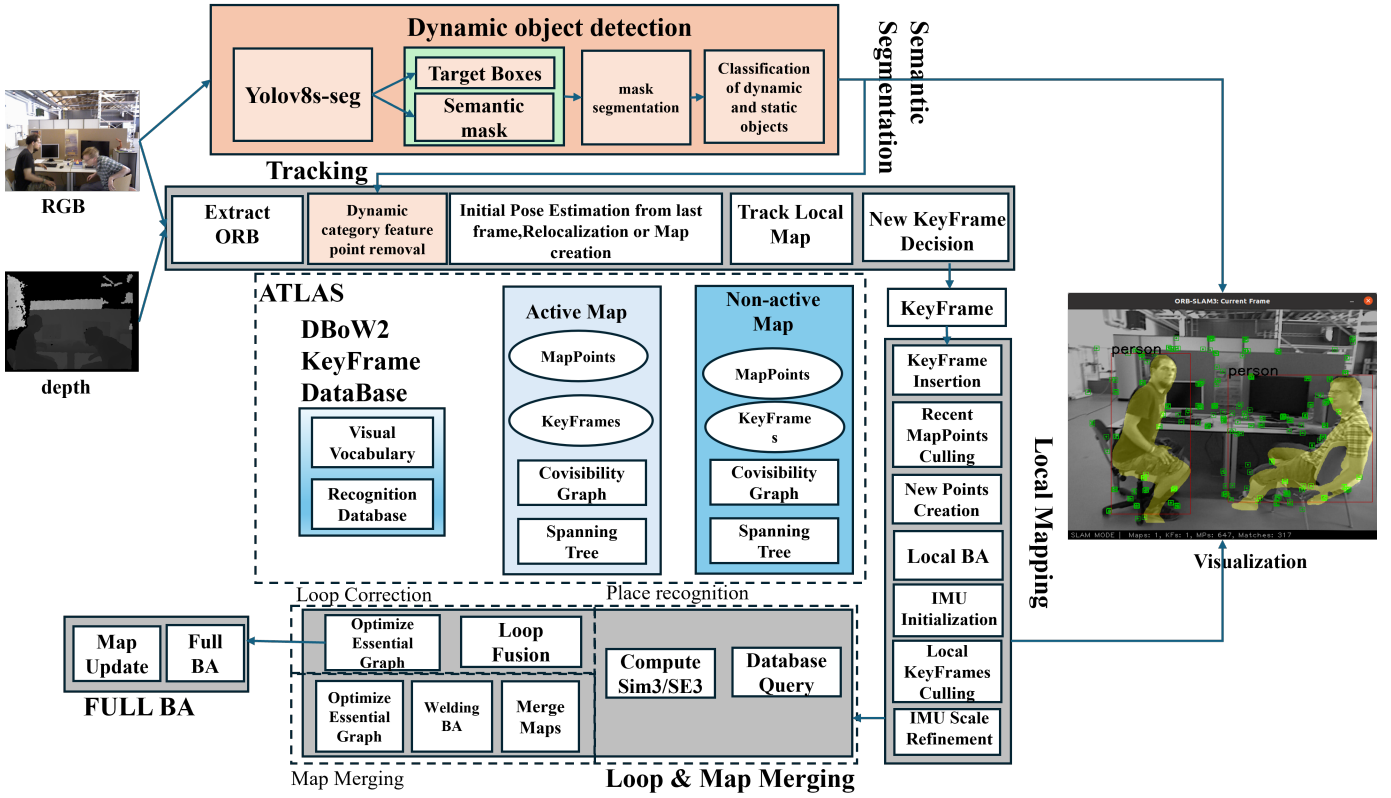


Figure 1: Overview of RY-SLAM Framework

error at the pixel level, without relying on discrete feature points. These methods perform well in low-texture environments but incur high computational costs, limiting real-time applicability. Kerl et al. [19] further enhanced direct SLAM by jointly minimizing photometric and geometric errors using grayscale and depth data. However, the inclusion of dense 3D mapping exacerbates latency issues.

ORB-SLAM3, the latest in the ORB series, improves tracking accuracy and robustness, particularly in complex scenarios [5]. It introduces two major innovations: (1) in visual-inertial odometry (VIO) mode, it integrates IMU pre-integration with joint bundle adjustment; (2) it employs a multi-map strategy for robust relocalization and loop closure.

B. Visual SLAM in Dynamic Scenes

To overcome the limitations of traditional SLAM in dynamic scenes, various systems have integrated semantic segmentation and motion analysis.

DS-SLAM [20], built on ORB-SLAM2, combines SegNet-based segmentation [21] with optical flow to filter dynamic feature points. While this improves stability, it only detects a limited set of objects and lacks robustness in complex environments. SOF-SLAM [22] combines dense optical flow with segmentation but suffers from cumulative drift due to the absence of global optimization.

DynaSLAM [23] integrates Mask R-CNN with multi-view geometry to identify dynamic regions. Though effective, its

high computational cost limits real-time use. DynaSLAM II [10] improves generality by detecting motion without relying solely on semantic labels but remains restricted to rigid objects.

Blitz-SLAM [11] uses BlitzNet [24] for segmentation and a two-stage motion detection pipeline. It identifies dynamic areas using semantic priors followed by optical flow residuals, significantly enhancing robustness. YOLO-SLAM [25] employs YOLOv3 for rapid detection but suffers from low accuracy and imprecise bounding boxes, reducing performance in static scenes.

To address these shortcomings, we introduce RY-SLAM, which embeds the YOLOv8s-seg model within the ORB-SLAM3 framework. Unlike previous YOLO-based methods, RY-SLAM generates fine-grained semantic masks instead of coarse bounding boxes. Additionally, a neighborhood-based strategy removes dynamic-region feature points, enhancing accuracy and robustness. The lightweight nature of YOLOv8s-seg ensures real-time performance, overcoming the computational burdens typically associated with deep-learning-based SLAM systems.

III. SYSTEM DESCRIPTION

A. Overview of RY-SLAM

The overview of our proposed RY-SLAM framework is shown in Fig.1. In our SLAM approach, modules extended from ORB-SLAM3 using YOLOv8s-Seg are highlighted with pink boxes, while the remaining components represent the original

ORB-SLAM3 structure. Our primary improvement focus on the tracking thread of ORB-SLAM3, ensuring efficient and smooth operation within a multi-threaded parallel framework. We integrated the YOLOv8 segmentation module into the SLAM system by instantiating the YOLOv8 segmentation class during system initialization, utilizing pointers to facilitate communication between threads. Specifically, we improve several key functions within the original ORB-SLAM3 framework: the initialization function `ORB_SLAM3::System()`, the tracking function `SLAM.TrackRGBD()`, the pose-acquisition function `Tracking::GrabImageRGBD()`, the ORB feature extraction function `ORBextractor::ComputeKeyPointsOctTree()`, and the visualization function `Viewer::Run()`.

Fig.2 shows the details of the improved RY-SLAM architecture in Fig.1, which includes the semantic segmentation framework we proposed and the main multi-threaded execution process of RY-SLAM. In the framework, the three threads we mainly improved and designed are YOLODetection, Tracking and Viewer threads. The YOLODetection thread utilizes the ONNXRuntime framework to deploy the YOLOv8 network, developed in Python, into the SLAM system implemented in C++. Additionally, we use this framework to enable GPU acceleration and optimizes the neural network inference process through graph optimization. To further enhance performance, we implement the WarmUp algorithm, which feeds three randomly initialized tensors into the network, thereby accelerating data transfer from memory to video memory and improving the overall inference speed.

During system initialization, the five main threads in the RY-SLAM system are launched and executed concurrently in a multi-threaded environment, with inter-thread communication facilitated through pointers. The primary function of the YOLODetection thread is to generate semantic masks for the camera key frames, specifically identifying dynamic areas and dynamic masks in each image. These key results are then transmitted to the Tracking and Viewer threads via pointers. The Tracking thread receives the semantic mask through the `GrabImageRGBD` function and shares it with other modules within the thread. Subsequently, the `ORBextractor` module uses the semantic mask to filter out dynamic feature points. For feature points located along the mask's edge, the module applies a neighborhood detection method for more refined processing, as shown in eq.(1)

$$\begin{cases} \text{reserve} & \text{num}(p(x_i) \neq 0) < 5 \\ \text{delete} & \text{num}(p(x_i) \neq 0) \geq 5 \end{cases} \quad (1)$$

Here, the num function counts the number of dynamic feature points within a 3×3 neighborhood around each feature point. If more than five dynamic points are detected, the feature point is classified as dynamic and removed; otherwise, it is retained. Additionally, the ORB feature point processing results, along with the semantic mask, are passed to the `ImShow` module in the Viewer thread. This module combines the results with the corresponding camera frame for visualization, allowing the system's operation to be displayed.

B. Semantic Segmentation Based on YOLOv8

The output procedure of the semantic segmentation architecture based on the YOLOv8 network is presented in Algorithm 1. An input image I is first processed by the neural network to produce preliminary feature maps a and m , corresponding to dynamic areas and dynamic masks, respectively. Detection bounding boxes in a are scaled to match the original image resolution, after which redundant bounding boxes are removed by calculating intersection ratios and applying additional filtering steps. This ensures each target object corresponds accurately to a single detection bounding box. Subsequently, the regions defined by these bounding boxes are extracted from m , and the resulting mask coefficients are multiplied by mask prototypes to generate preliminary mask instances. These preliminary masks undergo sigmoid normalization to map values to the range $[0, 1]$. Finally, applying a mask threshold converts these masks into binary form, yielding the finalized dynamic areas and dynamic masks.

Algorithm 1 Dynamic Area and Mask Calculation

```

1: Input: Image  $I$ 
2: By  $F(I) = (a, m)$ , calculate the dynamic area  $a$  and dynamic mask  $m$ 
3: Calculate confidence  $\eta$  by  $\varphi(a) = \eta$ 
4: if  $\eta > \tau_1$  then
5:   Through  $f(a) = (w, h, x, y)$  get detection box information from  $a$ 
6:   Restore the true resolution of the detection frame through formula  $\begin{cases} X = \frac{(x-dw)}{s} & W = \frac{w}{s} \\ Y = \frac{(y-dh)}{s} & H = \frac{h}{s} \end{cases}$ 
7:   Calculate the new detection box set according to the confidence  $g(X, Y, W, H|\eta) = B$ 
8:   while  $B \neq \emptyset$  do
9:     Select the current highest score frame  $b_{max} \in D$ 
10:    Calculate  $\text{IoU}(b_{max}, b_i) = \frac{\text{Area of Overlap}}{\text{Area of Union}}$ 
11:    if  $\text{IoU} > \tau_2$  then
12:      Delete  $b_i$  from  $B$ 
13:    end if
14:  end while
15:  Calculate segmentation features:  $M_{\text{instance}} = \sum_{i=1}^{32} (\text{Coefficient}_i \times \text{Prototype}_i)$ 
16:  if  $\text{Sigmoid}(M_{ij}) > \tau_3$  then
17:     $M_{ij} = 1$ 
18:  else
19:     $M_{ij} = 0$ 
20:  end if
21:  Output  $\text{DynamicArea} = D, \text{DynamicMask} = M$ 
22: end if

```

IV. EXPERIMENTAL RESULTS

In this section, we present the experimental results of RY-SLAM. The evaluation was conducted using the TUM-RGBD dataset, a standard benchmark widely utilized for assessing SLAM algorithms. We analyze the performance of our SLAM

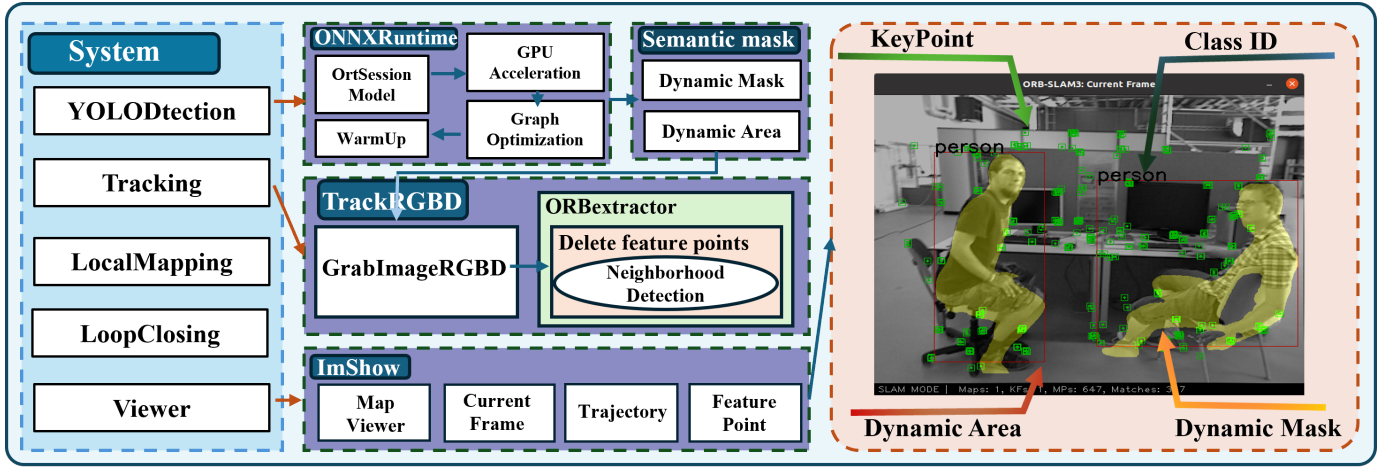


Figure 2: Details of RY-SLAM Semantic Segmentation Framework and Multi-Threaded Execution Process

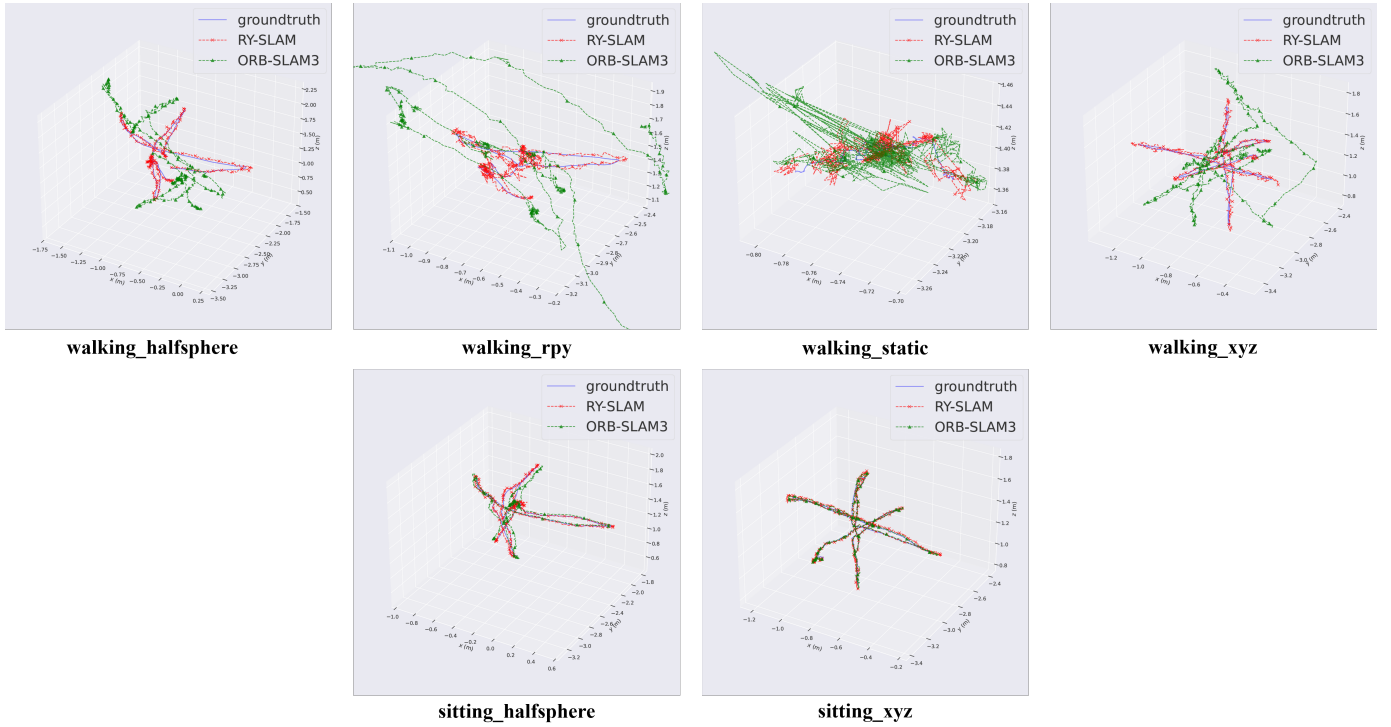


Figure 3: Comparison of estimated trajectory and true trajectory

algorithm by examining camera tracking trajectories, absolute trajectory errors, relative pose errors, and tracking time.

We selected six datasets from the TUM RGB-D dataset to experimentally evaluate the proposed RY-SLAM system. The TUM RGB-D dataset [26] is widely utilized for benchmarking visual SLAM, 3D reconstruction, and motion-capture algorithms. Specifically, we chose datasets designed to assess tracking performance in dynamic environments: fr3/walking_xyz, fr3/walking_static, fr3/walking_rpy, fr3/walking_halfsphere, fr3/sitting_xyz and fr3/sitting_halfsphere.

In the TUM dataset, the label “walking” denotes scenes with

moving pedestrians, whereas “sitting” refers to scenes with stationary individuals. The terms “xyz,” “static,” “rpy,” and “halfsphere” specify different types of camera motion within the environment. Each dataset contains RGB images, depth images, inertial measurement unit (IMU) data, ground-truth camera trajectories, and intrinsic calibration parameters for both the camera and IMU sensors.

Figure 3 presents a comparison of camera trajectories estimated by RY-SLAM and ORB-SLAM3 on selected sequences from the TUM RGB-D dataset. In each plot, the blue solid line denotes the ground-truth trajectory, the red dotted line

Table I: Results Of Absolute Trajectory Error(ATE)

Sequences	RY-SLAM		ORB-SLAM3		DS-SLAM		Dyna-SLAM		Improvement(%)	
	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.
walking_halfsphere	0.0235	0.0109	0.4033	0.0989	0.0303	0.0159	0.0296	0.0157	94.17%	88.98%
walking_rpy	0.0343	0.0180	0.7541	0.3274	0.4442	0.2350	0.0354	0.0190	95.45%	94.49%
walking_static	0.0065	0.0031	0.0236	0.0169	0.0081	0.0033	0.0068	0.0032	72.46%	81.95%
walking_xyz	0.0156	0.0073	0.4160	0.2240	0.0247	0.0161	0.0164	0.0086	96.26%	96.74%
sitting_halfsphere	0.0177	0.0077	0.0500	0.0179	0.0166	0.0077	0.0186	0.0086	64.54%	56.74%
sitting_xyz	0.0134	0.0062	0.0111	0.0054	0.0115	0.0056	0.0127	0.0060	-20.99%	-14.97%

with cross markers indicates the RY-SLAM trajectory, and the green dotted line with triangle markers represents the trajectory estimated by ORB-SLAM3.

As shown in Figure 3, the trajectory produced by ORB-SLAM3 (green) deviates significantly from the ground truth in the presence of dynamic objects. This degradation is especially evident in the walking_rpy dataset, where the system encounters tracking failure under combined dynamic motion and camera rotation. In contrast, RY-SLAM (red) yields trajectories that remain closely aligned with the ground truth, particularly in the walking_xyz and walking_halfsphere sequences, where trajectory error is minimal.

Under static conditions—such as the sitting series datasets—both systems perform comparably. In sitting_xyz, the trajectories estimated by RY-SLAM and ORB-SLAM3 nearly overlap with the ground truth, indicating high tracking accuracy in static scenes.

In summary, RY-SLAM, which augments ORB-SLAM3 with the YOLOv8s-Seg semantic segmentation network, significantly improves trajectory estimation in dynamic environments. Moreover, it retains competitive accuracy in static scenarios, confirming its robustness and adaptability across varying motion conditions.

We evaluate the performance of RY-SLAM against three existing SLAM systems—ORB-SLAM3, DS-SLAM, and Dyna-SLAM—on the TUM RGB-D dataset. The evaluation metrics include Absolute Trajectory Error (ATE) and Relative Pose Error (RPE). ATE quantifies the absolute deviation between the estimated and ground truth trajectories, while RPE measures the error in relative motion between consecutive frames.

Tables I, II, and III present the evaluation results, where bold values indicate the best performance among the four systems for each metric. Improvement percentages are calculated using Equation 2, defined as:

$$I = \frac{e_{orb} - e_{ry}}{e_{orb}} \times 100\% \quad (2)$$

where I denotes the improvement percentage, e_{orb} is the metric value obtained using ORB-SLAM3, and e_{ry} is the corresponding metric value from RY-SLAM. As shown in Table I, RY-SLAM significantly outperforms ORB-SLAM3 across all four dynamic sequences. In most cases, the improvement in root mean square error (RMSE) exceeds 90%, indicating a substantial gain in accuracy. Compared to DS-SLAM and Dyna-SLAM, RY-SLAM also achieves the best overall performance

on these dynamic datasets. The low RMSE values reflect RY-SLAM's high tracking precision, while the low standard deviation (S.D.) indicates strong stability across runs. On the two static datasets, although RY-SLAM does not perform the best, the performance gap is relatively small. This suggests that RY-SLAM maintains good adaptability even in static environments.

Tables II and III further confirm that RY-SLAM achieves consistently better results in both translational and rotational components of RPE across all dynamic scenarios. While ORB-SLAM3 achieves the best RPE performance in static scenes, RY-SLAM remains competitive and surpasses the other dynamic SLAM systems, further validating its robustness in a variety of environments.

A. Evaluation Of Processing Time

As shown in Table IV, RY-SLAM achieves an average tracking time of approximately 24 ms per frame, equivalent to a processing rate of around 40 frames per second. Throughout all experiments, viewer thread execution and visual rendering remained smooth and responsive, demonstrating that RY-SLAM effectively meets real-time performance requirements.

V. CONCLUSION

In this paper, we proposed RY-SLAM, a robust real-time visual SLAM system integrating a novel semantic segmentation architecture based on YOLOv8s-Seg into ORB-SLAM3. This approach effectively addresses two critical challenges: the limited robustness of traditional visual SLAM in dynamic environments and the inadequate real-time performance observed in neural network-based SLAM frameworks. Experimental results demonstrate that RY-SLAM achieves an average tracking time of approximately 24 ms per frame, fulfilling real-time operation requirements.

Nevertheless, RY-SLAM still presents significant potential for further improvement. Currently, the system is restricted to generating sparse maps, and experimental validations have been confined exclusively to indoor environments. Future work will explore extending the system's applicability to outdoor scenarios and enhancing its capability to produce precise dense maps.

REFERENCES

- [1] X. Duan, Q. Hu, P. Zhao, F. Yu, and M. Ai, "A low-drift and real-time localisation and mapping method for handheld hemispherical view lidar-imu integration system," *The Photogrammetric Record*, vol. 38, no. 182, pp. 176–196, 2023.

Table II: Results Of Translation Relative Pose Error(RPE)

Sequences	RY-SLAM		ORB-SLAM3		DS-SLAM		Dyna-SLAM		Improvement(%)	
	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.
walking_halfsphere	0.0140	0.0076	0.2166	0.1145	0.0297	0.0152	0.0284	0.0149	93.52%	93.39%
walking_rpy	0.0201	0.0127	0.5049	0.3077	0.1503	0.1168	0.0448	0.0262	96.02%	95.87%
walking_static	0.0061	0.0036	0.1165	0.1134	0.0102	0.0038	0.0089	0.0044	94.76%	96.83%
walking_xyz	0.0127	0.0075	0.2222	0.1136	0.0333	0.0229	0.0217	0.0119	94.27%	93.41%
sitting_halfsphere	0.0118	0.0065	0.0081	0.0044	0.0177	0.0084	0.0239	0.0120	-45.69%	-48.08%
sitting_xyz	0.0099	0.0052	0.0086	0.0044	0.0133	0.0069	0.0142	0.0073	-14.82%	-18.91%

Table III: Results Of Rotational Relative Pose Error(RPE)

Sequences	RY-SLAM		ORB-SLAM3		DS-SLAM		Dyna-SLAM		Improvement(%)	
	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.	RMSE	S.D.
walking_halfsphere	0.4148	0.2141	5.5333	3.3133	0.8142	0.4101	0.7842	0.4012	92.50%	93.54%
walking_rpy	0.5049	0.3077	6.6845	4.4043	3.0042	2.3065	0.9894	0.5701	92.45%	93.01%
walking_static	0.1746	0.0907	3.3217	2.2302	0.2690	0.1215	0.2612	0.1259	94.74%	95.93%
walking_xyz	0.3905	0.2737	5.5461	3.3482	0.8266	0.2826	0.6284	0.3848	92.96%	91.82%
sitting_halfsphere	0.3989	0.2096	0.3747	0.1978	0.6098	0.2850	0.7045	0.3488	-6.46%	-5.96%
sitting_xyz	0.3237	0.1663	0.3180	0.1649	0.5044	0.2623	0.5042	0.2651	-1.81%	-0.83%

Table IV: Time Of Processing

Sequences	s_half	s_xyz	w_half	w_rpy	w_static	w_xyz
Mean(s)	0.0247	0.0245	0.0250	0.0238	0.0241	0.0245
Median(s)	0.0246	0.0245	0.0249	0.0238	0.0239	0.0245

- [2] J. Li, R. Zhong, Q. Hu, and M. Ai, "Feature-based laser scan matching and its application for indoor mapping," *Sensors*, vol. 16, no. 8, p. 1265, 2016.
- [3] J. Li, P. Shi, Q. Hu, and Y. Zhang, "Qgore: Quadratic-time guaranteed outlier removal for point cloud registration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 9, pp. 11136–11151, 2023.
- [4] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5135–5142, IEEE, 2020.
- [5] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," vol. 37, no. 6, pp. 1874–1890.
- [6] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 3, pp. 611–625, 2017.
- [7] R. Mur-Artal and J. D. Tardos, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," pp. 1255–1262.
- [8] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "Orb-slam: A versatile and accurate monocular slam system," *IEEE transactions on robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [9] I. Abaspor Kazerouni, L. Fitzgerald, G. Dooley, and D. Toal, "A survey of state-of-the-art on visual SLAM," vol. 205, p. 117734.
- [10] B. Bescos, C. Campos, J. D. Tardos, and J. Neira, "DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM," pp. 5191–5198.
- [11] Y. Fan, Q. Zhang, Y. Tang, S. Liu, and H. Han, "Blitz-SLAM: A semantic SLAM in dynamic environments," vol. 121, p. 108225.
- [12] R. Li, S. Wang, Z. Li, and D. Gu, "Undeepvo: Monocular visual odometry through unsupervised deep learning," *Cornell University - arXiv, Cornell University - arXiv*, Sep 2017.
- [13] R. Li, S. Wang, and D. Gu, "Deepslam: A robust monocular slam

system with unsupervised deep learning," *IEEE Transactions on Industrial Electronics*, p. 3577–3587, Apr 2021.

- [14] T. Taketomi, H. Uchiyama, and S. Ikeda, "Visual slam algorithms: a survey from 2010 to 2016," *IPSP Transactions on Computer Vision and Applications*, Dec 2017.
- [15] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *Computer Vision – ECCV 2014, Lecture Notes in Computer Science*, pp. 834–849.
- [16] S. Bianco, G. Ciocca, and D. Marelli, "Evaluating the performance of structure from motion pipelines," *Journal of Imaging*, p. 98, Aug 2018.
- [17] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *2011 International Conference on Computer Vision*.
- [18] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison, "DTAM: Dense tracking and mapping in real-time," in *2011 International Conference on Computer Vision*.
- [19] C. Kerl, J. Sturm, and D. Cremers, "Dense visual SLAM for RGB-D cameras," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*.
- [20] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, "DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [21] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," pp. 2481–2495.
- [22] L. Cui and C. Ma, "SOF-SLAM: A Semantic Visual SLAM for Dynamic Environments," pp. 166528–166539.
- [23] B. Bescos, J. M. Facil, J. Civera, and J. Neira, "DynaSLAM: Tracking, Mapping and Inpainting in Dynamic Scenes," pp. 4076–4083.
- [24] N. Dvornik, K. Shmelkov, J. Mairal, and C. Schmid, "BlitzNet: A Real-Time Deep Network for Scene Understanding,"
- [25] W. Wu, L. Guo, H. Gao, Z. You, Y. Liu, and Z. Chen, "YOLO-SLAM: A semantic SLAM system towards dynamic environment with geometric constraint," vol. 34, no. 8, pp. 6011–6026.
- [26] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *Proc. of the International Conference on Intelligent Robot Systems (IROS)*.